PRACTICAL: 13

PHP JSON RESTful API & Asynchronous Single-Page CRUD Application using Fetch API

Objective:

To develop a RESTful JSON API in PHP that handles standard HTTP methods (GET, POST, PUT, DELETE) and to build an asynchronous Single-Page Application (SPA) frontend using JavaScript fetch(), async/await, dynamic DOM updating, and modal-based editing without full page reloads.

Theory:

Modern web applications decouple user interfaces from backend database operations using asynchronous client-server communication. Rather than triggering full browser page refreshes on every form submission, the frontend dispatches background HTTP requests via the Fetch API and dynamically updates the DOM upon receiving JSON payloads.

- • **RESTful Architecture & HTTP Verbs:** The backend api.php endpoint maps standard HTTP methods to CRUD operations: GET retrieves student data or filters by search query; POST creates a new student; PUT/POST updates an existing record; and DELETE removes records. OPTIONS handles preflight CORS checks.
- • **JSON Content Negotiation:** The server sets Content-Type: application/json; charset=UTF-8 and receives incoming JSON bodies via json_decode(file_get_contents('php://input'), true). It transmits standardized responses containing status, message, and record payloads.
- • **Asynchronous JavaScript & Promises:** ES6 fetch() utilizes Promises with async/await syntax to execute network calls asynchronously. The application continues running without blocking the browser UI thread while waiting for MySQL responses.
- • **Dynamic DOM Injection:** Data received from the JSON API is mapped directly into HTML table rows using JavaScript template literals, completely eliminating page flickers and retaining scroll positions.
- • **Modal Dialogs & Event Delegation:** Creating and editing student records is managed inside a responsive modal dialog. Event listeners on action buttons dynamically populate form inputs and switch states between Create and Update.
- • **Real-Time Debounced Search:** Keystrokes in the search bar trigger debounced Fetch requests to fetch('api.php?search=...') after a 300ms pause, optimizing network bandwidth while providing instant feedback.
- • **Non-Intrusive Toast Alerts:** Feedback banners render dynamically on the screen to inform the user of operation status (HTTP 200 OK or HTTP 400/500 errors) before automatically fading out.

Note: All database interactions in api.php use prepared statements with bind_param() to safeguard the REST API against SQL injection attacks.

Code:

api.php (RESTful Router & GET Handler)

```php
<?php
/**
 * Practical 13: PHP JSON RESTful API for Students & Events
 * Supports:
 * - GET: Fetch all or search students/events
 * - POST: Create new record
 * - PUT / POST (action=update): Update existing record
 * - DELETE / POST (action=delete): Delete record
 * Returns standardized JSON responses with status codes.
 */

header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With");

require_once __DIR__ . "/db.php";

$conn = getDbConnection();
$method = $_SERVER['REQUEST_METHOD'];

// Handle preflight
if ($method === 'OPTIONS') {
    http_response_code(200);
    exit;
}

if (!$conn) {
    http_response_code(500);
    echo json_encode(["status" => "error", "message" => "Database connection unavailable. Ensure MySQL is running."]);
    exit;
}

// Read JSON input body if application/json
$input = json_decode(file_get_contents("php://input"), true) ?: $_POST;

// 1. GET REQUEST: List or Search
if ($method === 'GET') {
    $search = trim($_GET['search'] ?? '');
    $id = (int)($_GET['id'] ?? 0);

    if ($id > 0) {
        $stmt = $conn->prepare("SELECT * FROM students WHERE id = ?");
        $stmt->bind_param("i", $id);
        $stmt->execute();
```

api.php (POST / PUT Student Record Handler)

```php
// 2. POST REQUEST: Create or (Update/Delete based on action)
if ($method === 'POST') {
    $action = $input['action'] ?? 'create';

    // Check if it's an update
    if ($action === 'update' || isset($input['id']) && (int)$input['id'] > 0 && $action !== 'delete') {
        $id      = (int)$input['id'];
        $roll_no = trim($input['roll_no'] ?? '');
        $name    = trim($input['name'] ?? '');
        $email   = trim($input['email'] ?? '');
        $course  = trim($input['course'] ?? '');
        $year    = trim($input['year'] ?? '3rd Year');
        $cgpa    = (float)($input['cgpa'] ?? 8.0);

        if (empty($roll_no) || empty($name) || empty($email) || empty($course)) {
            http_response_code(400);
            echo json_encode(["status" => "error", "message" => "All required fields must be supplied."]);
            exit;
        }

        $stmt = $conn->prepare("UPDATE students SET roll_no=?, name=?, email=?, course=?, year=?, cgpa=? WHERE id=?");
        $stmt->bind_param("ssssssdi", $roll_no, $name, $email, $course, $year, $cgpa, $id);

        if ($stmt->execute()) {
            logAuditAction("API Updated student #$id", 'api', 'api');
            echo json_encode(["status" => "success", "message" => "Student updated successfully via API."]);
        } else {
            http_response_code(500);
            echo json_encode(["status" => "error", "message" => "Database error: " . $stmt->error]);
        }
        $stmt->close();
        exit;
    }

    // Check if it's a delete via POST
    if ($action === 'delete') {
        $id = (int)($input['id'] ?? 0);
        if ($id <= 0) {
            http_response_code(400);
            echo json_encode(["status" => "error", "message" => "Invalid ID for deletion."]);
            exit;
        }

        $stmt = $conn->prepare("DELETE FROM students WHERE id = ?");
        $stmt->bind_param("i", $id);
        if ($stmt->execute()) {
            logAuditAction("API Deleted student #$id", 'api', 'api');
            echo json_encode(["status" => "success", "message" => "Student record deleted successfully via API."]);
        } else {
            http_response_code(500);
            echo json_encode(["status" => "error", "message" => "Database error: " . $stmt->error]);
        }
        $stmt->close();
```

practical13_ajax.php (SPA DOM Structure)

```
<main>
  <div class="card" style="margin-bottom: 20px;">
    <div class="card-header">
      <div>
        <h1 class="card-title">Asynchronous Student Management (Fetch API + JSON Backend)</h1>
        <p style="font-size: 0.85rem; color: var(--text-muted); margin-top: 4px;">
          All operations (List, Add, Update, Delete, Search) are performed via <code>api.php</code> asynchronously without page re
        </p>
      </div>
      <button class="btn btn-primary" id="openAddModalBtn">+ Add Student (AJAX)</button>
    </div>

    <!-- Live Search Bar -->
    <div class="flex-between flex-wrap" style="gap: 12px; margin-top: 10px;">
      <div style="flex: 1; min-width: 260px;">
        <input type="text" id="ajaxSearchInput" class="form-control" placeholder="Type to search dynamically (debounced)...">
      </div>
      <button class="btn btn-outline" id="refreshBtn">🔄 Refresh Data</button>
    </div>

    <!-- Toast Feedback -->
    <div id="apiFeedback" style="display:none; margin-top: 15px;" class="alert"></div>
  </div>

  <!-- Data Table Container -->
  <div class="card">
    <div class="card-header">
      <h3 class="card-title">Live Students Directory</h3>
      <span id="recordCountBadge" class="badge" style="background:var(--primary-light); color:var(--primary); padding:4px 8px; bor
    </div>

    <div class="table-responsive">
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Roll Number</th>
            <th>Full Name</th>
            <th>Email</th>
            <th>Course / Year</th>
            <th>CGPA</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody id="studentsTableBody">
          <tr>
            <td colspan="7" style="text-align: center; padding: 30px;">
              <div style="color: var(--text-muted);">🔄 Connecting to PHP JSON API...</div>
```

practical13_ajax.php (Async Fetch & DOM Logic)

```
            <label for="modalCourse">Course *</label>
            <select id="modalCourse" class="form-control" required>
              <option value="B.Tech AIML">B.Tech AIML</option>
              <option value="B.Tech CE">B.Tech CE</option>
              <option value="B.Tech IT">B.Tech IT</option>
              <option value="B.Tech EC">B.Tech EC</option>
            </select>
          </div>

          <div class="form-group">
            <label for="modalYear">Year *</label>
            <select id="modalYear" class="form-control">
              <option value="1st Year">1st Year</option>
              <option value="2nd Year">2nd Year</option>
              <option value="3rd Year" selected>3rd Year</option>
              <option value="4th Year">4th Year</option>
            </select>
          </div>
        </div>

        <div class="form-group">
          <label for="modalCgpa">CGPA</label>
          <input type="number" step="0.01" min="0" max="10" id="modalCgpa" class="form-control" value="8.50">
        </div>

        <div style="display:flex; justify-content: flex-end; gap: 10px; margin-top: 15px;">
          <button type="button" class="btn btn-outline" data-modal-close>Cancel</button>
          <button type="submit" id="saveStudentBtn" class="btn btn-primary">Save Changes</button>
        </div>
      </form>
    </div>
  </div>

  <footer>
    <p>StudentHub &copy; <?= date("Y") ?> CHARUSAT FTE | Practical 13 Asynchronous Fetch API & JSON API</p>
  </footer>

  <script>
    const API_URL = 'api.php';
    const tableBody = document.getElementById('studentsTableBody');
    const recordCountBadge = document.getElementById('recordCountBadge');
    const feedbackBox = document.getElementById('apiFeedback');
    const studentModal = document.getElementById('studentModal');
    const studentForm = document.getElementById('ajaxStudentForm');
    const searchInput = document.getElementById('ajaxSearchInput');

    function showFeedback(msg, type = 'success') {
      feedbackBox.className = `alert alert-${type}`;
      feedbackBox.textContent = msg;
      feedbackBox.style.display = 'block';
      setTimeout(() => {
        feedbackBox.style.display = 'none';
      }, 4000);
    }

    // 1. Fetch and Render Students
```

Raw JSON Response from api.php (Inspected in Browser DevTools / Network Tab)

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=UTF-8
Access-Control-Allow-Origin: *

{
  "status": "success",
  "message": "Student created successfully",
  "student_id": 17,
  "data": {
    "roll_no": "D26AIML091",
    "name": "Jay Dangar",
    "email": "d26aiml091@charusat.edu.in",
    "course": "B.Tech CSE (AI & ML)",
    "cgpa": 9.25
  }
}
```

Learning Outcome:

Students developed and deployed an enterprise-grade decoupled web module combining a PHP RESTful JSON API and a modern JavaScript Single-Page Application. They gained practical experience handling multiple HTTP verbs (GET, POST, PUT, DELETE), decoding raw JSON payloads, executing asynchronous network requests via fetch() and async/await, dynamically re-rendering DOM elements, and implementing debounced live search with real-time toast user feedback.

References:

- • **MDN Web Docs – Using the Fetch API:** https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- • **PHP Manual – json_encode() & json_decode():** https://www.php.net/manual/en/function.json-encode.php
- • **RESTful API Design Best Practices:** https://restfulapi.net/http-methods/